Foundations in Data Engineering

Notes by Marco Galletti

Lecturer: Jana Giceva

Technical University of Munich
Winter Semester 2026/27

Contents

1 Basic building blocks: data formats and the command line **2**

1.1 Data exchange formats 2

1.2 Streams and pipes 3

1.3 Command-line tools as relational operators 3

A Command reference **6**

1 Basic building blocks: data formats and the command line

Given a large, unknown dataset, the first questions are what the data look like, which tool suits the analysis, and what hardware it requires. This section covers how datasets are encoded for exchange, and how the standard Unix command-line tools give a first impression of a dataset and answer simple queries on it without writing a program.

1.1 Data exchange formats

Datasets are exchanged as plain text (CSV and its variants), as text encoding semi-structured data (XML, JSON), in open binary formats such as **Parquet**, or in proprietary binary formats; the latter are poorly suited to exchange, since only the vendor's software reads them. Faced with an unknown file, the tool `file` guesses its format from its content. Below, the same two records (name, address, job) are written in each text format.

CSV. A **CSV** (comma separated values) file stores one record per line, its fields separated by a delimiter. The delimiter is configurable: with a tab one gets **TSV**. The format is simple, human readable and extremely widespread, but full of subtleties because it is weakly standardised: a field that contains the delimiter must be quoted, and a quote character inside a quoted field is doubled (the value `a"b` is written `"a""b"`). Records are flat: a row of fields, with no nesting.

```
Sherlock Holmes,221B Baker Street,Detective
James Moriarty,Reichenbach Falls,Villain
```

Written as "surname, first name", the first field would contain the delimiter and need quotes:

```
"Holmes, Sherlock",221B Baker Street,Detective
```

XML. **XML** encodes (semi-)structured data as a tree of nested elements delimited by opening and closing tags, possibly with attributes. In the example a root element contains one empty element `<person ... />` per record, whose fields are attributes:

```
<characters>
<person name="Sherlock Holmes" address="221B Baker Street" job="Detective"/>
<person name="James Moriarty" address="Reichenbach Falls" job="Villain"/>
</characters>
```

XML is better standardised than CSV and handles nested data, but it is very verbose (every value is wrapped in markup), human readable yet tedious to write, and its full specification is very complex. Advanced features such as **XML Schema** (a description of the allowed document structure) exist but are rarely used; its popularity is declining.

JSON. **JSON** (JavaScript Object Notation) is a much simpler text format. A JSON document is always an **array** `[v1, v2, ...]` or an **object** `{"key": value, ...}`: items are separated by commas, keys are strings in double quotes followed by a colon and a value, and values may again be arrays or objects, so data can be nested. There are only six data types: string, number, boolean, null, object and array. The two records form an array of two objects:

```
[{"name": "Sherlock Holmes", "address": "221B Baker Street", "job": "Detective"},
 {"name": "James Moriarty", "address": "Reichenbach Falls", "job": "Villain"}]
```

JSON is less verbose and easier to write than XML, and increasingly popular.

Remark. The most common JSON errors violate exactly these rules: unquoted keys, strings in single instead of double quotes, and a trailing comma after the last item of an array or object.

Format	Encoding	Structure	Verbosity	Standardisation and use
CSV/TSV	text	flat rows	low	weak, many subtleties; very popular
XML	text	nested	very high	well standardised, complex spec; declining
JSON	text	nested	moderate	simple spec; growing

Table 1: The text-based exchange formats compared.

1.2 Streams and pipes

Text files are popular because they are human readable and easy to process. The Unix command-line tools are available by default on every Unix/Linux system; each is simple, but they are surprisingly powerful in combination. They are ideal to get a first impression, to explore the data and to run simple analyses without code; for the real analysis one usually moves on to a more suitable tool.

Standard streams. Every process has three standard streams, identified by number: **stdin** (0) for input, **stdout** (1) for output and **stderr** (2) for error messages. A **pipe** feeds the stdout of one command into the stdin of the next, and **redirection** connects a stream to a file instead (fig. 1); the **null device** `/dev/null` discards what is written to it, so redirecting stderr there suppresses error messages. Since errors travel on their own stream, they do not mix with the data flowing through a pipeline. Most tools read the files given as arguments, or stdin if there are none.

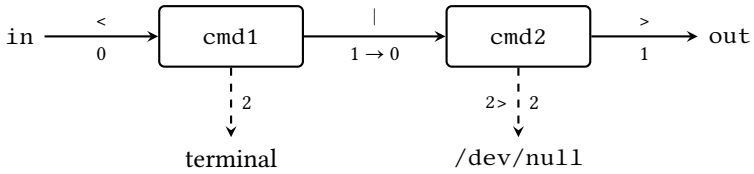


Figure 1: Streams of `cmd1 < in | cmd2 2> /dev/null > out`; stderr of `cmd1` still goes to the terminal.

Streaming large and compressed files. A tool in a pipeline sees its input as a stream of lines, so files are processed without being loaded or copied. In particular, the `*cat` variants of the compressors (gzip: Lempel–Ziv; bzip2: Burrows–Wheeler transform; xz: LZMA2) decompress to stdout, and a compressed dataset can be analysed without ever writing the decompressed data to disk. Pagers and prefix/suffix tools (`less`, `head`, `tail`) and counts (`wc`) give the first impression of a file: what a record looks like, how many there are.

1.3 Command-line tools as relational operators

A line-based text file can be read as a relation: each line is a tuple, and the fields separated by a delimiter (whitespace, a comma, a tab) are its attributes. The tools then act as relational operators on a stream of tuples, and a pipeline composes them into a query (table 2 and fig. 2).

Operator	Tool
selection (keep the lines satisfying a predicate)	`grep` (regular expression), `grep -v` (negated)
projection (keep some attributes)	`cut` (fields), `grep -o` (matched part)
sorting, duplicate elimination	`sort`, `uniq`
grouping with count	`sort`, then `uniq -c`
top-k, minimum, maximum	`sort`, then `head` or `tail`
join (inner, outer, anti)	`join` on sorted inputs
map (rewrite each line)	`sed`
aggregation (count, sum, average)	`wc -l`, `awk`
random sample	`shuf`, then `head`

Table 2: Command-line tools seen as relational operators; their flags are collected in section A.

Selection with regular expressions. A **regular expression** (regex) describes a set of strings. `grep` keeps the lines that contain at least one substring matched by the regex (selection), `grep -v` the lines that contain none (negated selection), and `grep -o` prints only the matched substrings, one per line (projection onto a pattern-defined part of the line); `sed` uses the same syntax for substitutions. A regex is built from the following pieces (table 3): ordinary characters match themselves; metacharacters have a meaning and match literally only when escaped with a backslash; a **quantifier** applies to the single item just before it (a character, a class or a group).

	Regex	Matches
characters	`a`, `.`, `\.`	the letter `a`; any single character; a literal dot
classes	`[abc]`, `[a-z0-9_]`	one character from the set (ranges allowed)
	`[^abc]`	one character *not* in the set
	`[[:space:]]`, `[[:digit:]]`	one whitespace character; one digit
quantifiers	`r*`, `r+`, `r?`	≥ 0, ≥ 1, 0 or 1 repetitions of `r`
	`r{n}`, `r{n,}`, `r{n,m}`	exactly n, at least n, between n and m
anchors	`^`, `$`	start, end of the line (match no character)
groups	`(r)`, `r|s`, `\1`	group; `r` or `s`; the text matched by group 1

Table 3: Regular expression syntax (extended form).

Basic and extended syntax, greediness. `grep` and `sed` use the **basic** syntax by default, in which `+ ? { } ( ) |` are ordinary characters and the operators must be escaped (`\{n,\}`, `\(r\)`; GNU also accepts `\+ \? \|`); `grep -E` (and `sed -E`) switch to the **extended** syntax of table 3. A regex is not anchored: `\.jpg` matches any line containing `.jpg`, and `^` or `$` are needed to pin it to the start or end. Quantifiers are **greedy**, i.e. they take the longest possible match, so `user=.*$` extracts everything from `user=` to the end of the line.

Remark. On the command line the regex is put in single quotes, so that the shell does not interpret `*`, `$` or `|` itself. The shell has its own patterns (**globs**) for file names, with a different meaning of `*`: the glob `*.html` ("any string, then `.html`") corresponds to the regex `^.*\.html$`, whereas in a regex `*` repeats the item before it.

Example 1.1 (Reading regexes)**.** `img[0-9]*\.jpg` matches `img` followed by zero or more digits and `.jpg`: it matches the lines `img.jpg`, `img7.jpg`, `img123.jpg`, and also `myimg12.jpg.bak` (the regex is not anchored), but not `imgA.jpg` (a letter is not a digit) nor `img5xjpg` (`\.` requires a dot). The regex `^warning` with `grep -v` keeps the lines that do not start with `warning`.

Example 1.2 (Extracting e-mail addresses)**.** With `grep -o -i -E`, the regex

$$\underbrace{\texttt{[_.a-z0-9]+}}_{\text{local part}}\ \texttt{@}\ \underbrace{\texttt{[a-z0-9]+}}_{\text{domain}}\ \underbrace{\texttt{\textbackslash.}}_{\text{dot}}\ \underbrace{\texttt{[a-z]\{2,\}}}_{\text{top-level domain}}$$

prints each e-mail address of the input on its own line: one or more letters, digits, underscores or dots, then `@`, one or more letters or digits, a literal dot and at least two letters (`-E` because of `+` and `{2,}`, `-i` so that the classes also cover capital letters). On `Ada.Lovelace@Example.ORG` it returns the whole address. It is a simplification of the full standard: the domain class contains neither dots nor hyphens, so from `x@in.tum.de` it extracts only `x@in.tum`, and `foo@mail-server.com` is not found at all.

Order matters. `sort` compares lexicographically unless told to compare numerically (so `10` comes before `9`), and it works on inputs larger than main memory. Its output is the required input of `uniq` and `join`: `uniq` only merges *adjacent* equal lines, and `join` merges two files by scanning both in order. Grouping is therefore sort-based: sorting brings equal values together, then `uniq` counts each run. The lines "occurring exactly once" (`uniq -u`) are not the distinct lines (plain `uniq`).

Top-k and frequencies. A **top-k query** is a sort followed by taking a prefix or a suffix; $k = 1$ gives the minimum or the maximum. Composed with grouping it yields the k most frequent values of an attribute (fig. 2).

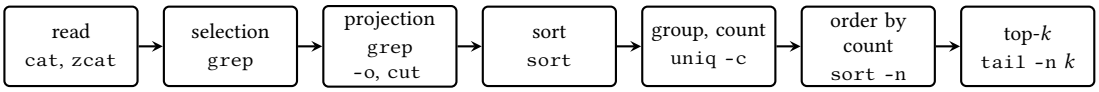


Figure 2: A pipeline computing the k most frequent values of an attribute, stage by stage.

Join. `join` pairs the lines of two sorted files with equal join field (an inner join). Keeping in addition the unmatched lines of the first, the second or both files gives the left, right and full **outer join**; keeping *only* the unmatched lines of one file gives the **anti-join**, which answers "which values of one list never occur in another". For heavier use of joins a more powerful tool is the better choice.

Map and aggregation. `sed` rewrites each line by substituting the matches of a regex; the replacement may refer to the whole match (`&`) or to the groups of the regex (`\1`, `\2`, …). For instance, the substitution (basic syntax)

```
s/IMG_\([0-9]*\)\.JPG/image\1.jpg/
```

renames `IMG_0042.JPG` to `image0042.jpg`. Replacing whitespace by newlines puts one word per line, which reduces word-level questions (e.g. word frequencies) to the line-level operators above. `awk` runs a small program in one pass: an initialisation, a block executed on every line (with access to its fields), and a final block. Writing a_i, b_i for the first two fields of line $i = 1, \dots, n$, this computes aggregates such as

$$n, \qquad \sum_{i=1}^{n} b_i, \qquad \frac{1}{n}\sum_{i=1}^{n} b_i, \qquad \sum_{i\colon a_i > 10} b_i,$$

by accumulating a sum and a count in variables and printing them (or their ratio) at the end.

Sampling. `shuf` outputs a random permutation of its input, so a prefix of it is a uniform random sample without replacement. A plain prefix of the file is not: files are often ordered (by time, by key), so their first lines are not representative. Shuffled inputs also serve for performance tests.

Limits. These tools stream their input and `sort` works beyond main memory, so they scale to large files and are ideal for a first look and simple queries. They operate on lines of text, however: they know nothing about quoting (`cut` splits a quoted CSV field containing the delimiter) or nesting, and anything beyond simple joins and aggregates calls for more powerful tools.

A Command reference

The commands and flags of the command-line tools, grouped by the relational operator they implement (section 1.3).

Streams

`a | b`	stdout of a into stdin of b
`cmd > f`	stdout to file `f`
`cmd < f`	stdin from file `f`
`cmd 2> /dev/null`	discard stderr

Stream ids: 0 stdin, 1 stdout, 2 stderr. `cmd f` = `cat f | cmd`.

Read and inspect

`cat f1 f2`	concatenate to stdout
`cat -v f`	show non-printing bytes
`zcat bzcat xzcat`	decompress to stdout (gzip, bzip2, xz)
`... | less`, `-S`	page through; `-S` chops long lines
`head -n k`	first k lines (also `head -k`)
`tail -n k`	last k lines
`tail -n +k`	from line k on (skip $k-1$)
`head -n -k`	all but the last k (GNU only)
`wc`, `-l`, `-c`	lines, words, bytes; lines only; bytes only

Selection and extraction: grep

`grep 're' f...`	lines matching `re`
`-v`	non-matching lines (negation)
`-i`	ignore case
`-o`	only the matched part, one per line
`-E`	extended regex (`+ ? {}` work)
`-a`	treat binary input as text

Regular expressions

`.`, `\.`	any character; a literal dot
`[a-z0-9_]`, `[^abc]`	one character of / not of a class
`[[:space:]]`, `[[:digit:]]`	whitespace; digit
`r*`	zero or more `r`
`r+`, `r?`	one or more; zero or one (`-E`)
`r{n}`, `r{n,}`, `r{n,m}`	n; $\geq n$; n to m (`-E`)
`^`, `$`	start, end of line
`(r)`, `r|s`, `\1`	group; alternative; back-reference (`-E`)
`\(r\)`, `\{n,\}`	the same in basic syntax

Quote the regex: `'...'`. Greedy: `.*` takes the longest match.

Projection: cut

`cut -f 1,3`	fields 1 and 3 (delimiter: tab)
`cut -f 2-4 -d ','`	fields 2 to 4, comma separated
`cut -c 1-10`	characters 1 to 10

Ignores CSV quoting.

Sorting: sort

`sort`	lexicographic (`10` before `9`)
`-n`	numeric
`-k f`	key from field f (`-k f,f`: only field f)
`-t ','`	field separator
`-u`	drop duplicates

Works beyond main memory.

Grouping: uniq (sorted input!)

`uniq`	one copy per line (distinct)
`uniq -c`	prefix each line with its count
`uniq -d`	lines occurring more than once
`uniq -u`	lines occurring exactly once

Join: join (both inputs sorted on the join field!)

`join f1 f2`	inner join on field 1
`-1 f -2 g`	join field f of `f1`, g of `f2`
`-t ','`	delimiter
`-a 1` / `-a 2`	left / right outer join
`-a 1 -a 2`	full outer join
`-v 1` / `-v 2`	left / right anti-join (unmatched only)

Map: sed

`sed 's/re/new/'`	replace first match per line
`s/.../.../g`	replace all matches
`s/.../.../I`	ignore case (GNU)
`&`, `\1`	in new: whole match, group 1
`-e '...' -e '...'`	several commands
`s:a/b:a_b:`	any delimiter instead of `/`

Aggregation: awk

`awk 'BEGIN { init } { per line } END { final }'`
`$1`, `$2`, ...: fields (whitespace separated).

`{ n++ } END { print n }`	count
`{ s+=$2 } END { print s }`	sum of field 2
`{ s+=$2; n++ } END { print s/n }`	average of field 2
`{ if ($1 > 10) s+=$2 }`	conditional sum

Sampling: shuf

`shuf | head -n k`	random sample of k lines

Patterns

count matching lines	`grep 're' f | wc -l`
distinct values	`cut -f 2 f | sort -u`
k most frequent	`sort | uniq -c | sort -n | tail -n k`
lines seen once	`sort | uniq -u`
min / max	`sort -n | head -n 1` / `tail -n 1`
in A but not in B	`sort A > a; sort B > b` `join -v 1 a b`
one word per line	`sed 's/[[:space:]]/\n/g'` (GNU)

Pitfalls

Sort before `uniq` and `join`. Sort counts with `-n`. Without `-E`, `+` is a literal plus. Binary bytes in a text file: `grep -a`. `uniq -u` ≠ distinct.